

Week 2

Classificatie met Support Vector Machines (SVM)

Herhalen

Vorige week: KNN

- **Voordeel:** zeer snelle training (lazy learner — enkel data opslaan)
- **Nadeel:** bij grote datasets **trage classificatie**, omdat er veel berekend moet worden voor een nieuw datapunt

 Vandaag bekijken we een classifieer die **trager traint**, maar **zeer snel classificeert!**

Large Margin Classification

Startprobleem: lineair separabele data

- We willen een classificatie die de twee klassen (bv. gele en blauwe bloemen) van elkaar onderscheidt
- Deze moet **lineair** zijn
 - Betekent later **heel snelle classificatie**

flowchart LR

```
Data["Dataset<br/>(2 klassen)"] --> Split["Zoek scheidingslijn"]
```

```
Split --> Vraag{"Lineair<br/>separeerbaar?"}
```

```
Vraag -- Ja --> SVM["✅ Lineaire classifieer"]
```

```
Vraag -- Nee --> Later["→ Kernels (later in de les)"]
```

Enkele pogingen

Er zijn veel mogelijke scheidingslijnen — welke is de beste?

flowchart LR

A["Lijn 1
(dicht bij klasse A)"] --> Q{" ? Welke
lijn?"}

B["Lijn 2
(dicht bij klasse B)"] --> Q

C["Lijn 3
(midden, ruime marge)"] --> Q

Q --> Best["→ Best: grootst mogelijke marge"]

Large margin classification

- 'Beste' optie:
 - De scheiding tussen de twee uiterste cases is **zo groot mogelijk**
 - = **large margin classification**
- Dit lukt helaas **niet altijd** (zie soft margin)

Large margin classification — in sklearn

```
from sklearn.svm import SVC

svm_clf = SVC(kernel="linear", C=0.25)
svm_clf.fit(X, y)
```

flowchart LR

```
A["Model maken<br/>SVC(kernel='linear', C=...)"] --> B["fit(X, y)"]
B --> C["Leert scheidingslijn<br/>+ maximale marge"]
```

De marge wordt bepaald door enkel de punten op de rand — de support vectors!



Code om decision boundary te tonen

```
def plot_svc_decision_boundary(svm_clf, xmin, xmax):
    w = svm_clf.coef_[0]
    b = svm_clf.intercept_[0]
    # Op het beslissingsvlak:  $w_0 \cdot x_0 + w_1 \cdot x_1 + b = 0$ 
    #  $\Rightarrow x_1 = -w_0/w_1 \cdot x_0 - b/w_1$ 
    x0 = np.linspace(xmin, xmax, 200)
    decision_boundary = -w[0] / w[1] * x0 - b / w[1]
    margin = 1/w[1]
    gutter_up = decision_boundary + margin
    gutter_down = decision_boundary - margin
    svcs = svm_clf.support_vectors_
    plt.plot(x0, decision_boundary, "k-", linewidth=2, zorder=-2)
    plt.plot(x0, gutter_up, "k--", linewidth=2, zorder=-2)
    plt.plot(x0, gutter_down, "k--", linewidth=2, zorder=-2)
    plt.scatter(svcs[:, 0], svcs[:, 1], s=180, facecolors='#AAA', zorder=-1)
```

Code ter **illustratie**, niet te kennen, maar mag je altijd gebruiken indien nodig.

Soft Margin Classification

Wat met niet-separabele data?

- Perfect scheiden is **niet mogelijk!**
- **Compromis:**
 - Zo breed mogelijke marge
 - Zo weinig mogelijke 'overtredingen' van die marge

```
flowchart LR
    Doel["SVM-doel"] --> M["Maximale marge"]
    Doel --> F["Minimale overtredingen"]
    M --> Balans{"Afwegen"}
    F --> Balans
    Balans --> C["→ Parameter C"]
```

Soft-margin classification

Hoe kleiner C , hoe meer 'fouten' er worden getolereerd:

- Lage $C \rightarrow$ risico **underfitting** (te tolerant)
- Hoge $C \rightarrow$ risico **overfitting** (te streng)

C	Marge	Fouten getolereerd	Risico
klein	breed	veel	underfitting
groot	smal	weinig	overfitting

Kies C via **cross-validatie**, niet "op het gevoel" 

Niet-lineaire data: Kernels

Voorbeeld: moons dataset

- Bevat 2 categorieën
- Is niet separabel, maar er zit wel een 'structuur' in

flowchart TD

Moon["🌙 Moons dataset"] --> Q{"Lineair SVM?"}

Q --> Fit["❌ Duidelijk geen goede fit"]

Eerste poging

- Lineair SVM
- Duidelijk geen goede fit

Tweede poging: polynomiale features

- Lineair SVM
- Maar nu met **nieuw toegevoegde features**, namelijk **polynomen tot de 3e graad**
- D.w.z. er zijn nieuwe **kolommen** in de dataset toegevoegd (feature engineering)

Derde poging: similariteitsfuncties

- SVM
- Maar nu met andere nieuw toegevoegde features, namelijk **similariteitsfuncties**
- Hier kijken we naar punten die **'op mekaar lijken'**

flowchart LR

```
A["Poging 1<br/>Lineair SVM"] -->|✗| B["Poging 2<br/>+ polynomiale features"]
B -->|✓| C["Poging 3<br/>+ similariteitsfuncties"]
C -->|✓| D["Kernel!"]
```

Wat doet die kernel?

- **Doel:** zorgen dat onze dataset in een **hogere dimensie** wél separabel is
- Concreet voor de moons dataset: je wil minstens **een extra dimensie** zodat er nu wel een **lineaire scheiding** (een vlak) kan worden gemaakt
- Eens de kernel gekozen, doet de classifier **zelf al het andere werk!** Hij werkt dan alsof hij een **lineaire classifier in hogere dimensie** is

Kernel Trick

flowchart LR

```
A["Data<br/>(niet-separabel<br/>in 2D)"] -->|"kernel (transformatie)"| B["Hogere dimensie<br/>(wel separabel)"]  
B --> C["Lineaire scheiding<br/>(vlak)"]  
C -->|"... komt overeen met"| D["Non-lineaire grens<br/>in originele 2D"]
```

- **Waarom is dit efficiënter?**
 - De kernel (een non-lineaire operator) wordt **niet uitgerekend**, maar enkel de **inproducten** tussen die features
 - Dankzij geavanceerde wiskunde kan dit efficiënt gebeuren

Wanneer SVM gebruiken?

- Niet-lineaire datasets
 - In de praktijk is bijna niets 'vanzelf' lineair
- Niet al te grote datasets
- Ze zijn onderhevig aan **scaling** — gebruik zeker een **StandardScaler**

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

svm_clf = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1))
```

Evaluatiesystemen

Hoe een model evalueren?

	Regressie	Classificatie
Standaard	R^2	Cross-validatie
Fouten	MAE / MSE / RMSE	Confusion matrix (zeker bij scheve labelverdeling), Precision, Recall, F1
Aandachtspunt	geen specifieke	multilabel classificatie + verdeling van de labels

Cross validation

- K-fold cross-validatie: data in K delen, elke deel één keer als testset

```
flowchart LR
    Data["Dataset"] --> S["Split in K folds"]
    S --> F1["Fold 1 = test<br/>rest = train"]
    S --> F2["Fold 2 = test<br/>rest = train"]
    S --> FK["..."]
    F1 & F2 & FK --> Avg["Gemiddelde score"]
    Avg --> Eind["→ Robuuste evaluatie"]
```

- Meer details: scikit-learn.org/stable/modules/cross_validation.html

Confusion Matrix

- Geeft per klasse aan wat het **echte label** (rij) en het **voorspelde label** (kolom) is
- Daardoor zie je meteen of een bepaalde klasse **slechter wordt voorspeld** (zoals hier de *versicolor*)
- In het ideale geval bevat de **diagonaal** zoveel mogelijk data 

```
flowchart LR
    subgraph CM["Confusion matrix"]
        direction LR
        E["Echt label"] --> R["Rij"]
        P["Voorspeld label"] --> K["Kolom"]
        D[" Diagonaal = correct"]
    end
end
```

Precision, Recall, F1 score

Statistieken die je uit de confusion matrix kan halen:

- **Precisie:** $P = \frac{TP}{TP + FP}$ (van alles wat we positief noemden, hoeveel was terecht?)
- **Recall:** $R = \frac{TP}{TP + FN}$ (van alle echt positieven, hoeveel vonden we?)
- **F1-score:** $F_1 = \frac{2PR}{P + R}$ (harmonisch gemiddelde)

Meer details: en.wikipedia.org/wiki/Confusion_matrix

Hyperparameters & Pipelines

GridSearchCV / RandomizedSearchCV

- **Doel:** automatisch de beste waarde van parameters vinden
- In ons geval 'C':

```
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100]}  
grid_search = GridSearchCV(svm_clf, param_grid, cv=5, scoring='accuracy')
```

GridSearchCV gebruiken

- De GridSearchCV moet je dan **fitten op je data**
- Daarna vind je de beste parameter in het `best_estimator_` object:

```
grid_search.fit(X_train, y_train)
grid_search.best_estimator_
```

flowchart LR

```
A["param_grid<br/>(waarden voor C)"] --> B["GridSearchCV<br/>(cv=5)"]
B --> C["fit(X_train, y_train)"]
C --> D["best_estimator_"]
```

Wat met niet-numerieke data?

- Typisch **labels, categorieën, ...**
- Geen onderling verband
- Gebruik **One-Hot encoding**
- Vervang de niet-numerieke categorieën door de geëncodeerde kolommen

```
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder(handle_unknown='ignore', sparse=False, drop='first')
encoded_categorical = enc.fit_transform(X_categorical)
```

We kunnen dit ook direct doen met een **preprocessor** in een pipeline.

Pipelines

- Intussen moet je een heel aantal dingen doen:
 - Een **preprocessing scaler** gebruiken
 - **Data splitting** in train-test
 - Je **model trainen**
 - Eventueel **grid search** doen
 - ...

```
flowchart LR
    X["X"] --> Sc["StandardScaler"]
    Sc --> Enc["OneHotEncoder"]
    Enc --> M["SVC model"]
    M --> P["Voorspelling"]
```

- Dit kan je allemaal in een **Pipeline** steken → voorkomt ook **data leakage** ✓

Samenvatten

Beste praktijken (Week 2)

- **Schaal je data (StandardScaler) vóór SVM** — SVM's zijn onderhevig aan scaling
- Stem C af via (Grid/Randomized)SearchCV met cross-validatie
 - lage C = toleranter, hoge C = meer overfitting-risico
- Gebruik **Pipelines** zodat preprocessing en model altijd dezelfde stappen doorlopen
- Evaluer classificatie met **confusion matrix + precision/recall/F1**, niet enkel accuracy
- Encode categorische features met **One-Hot encoding** (`handle_unknown='ignore'`)

Kernpunten

- SVM zoekt de **grootst mogelijke marge**; soft margin (**C**) tolereert overtredingen
- **Kernels** maken niet-lineaire data lineair separabel in hogere dimensie, efficiënt via de **kernel trick**
- Evaluatie: R^2 / MAE / MSE / RMSE voor regressie; confusion matrix, precision/recall/F1, cross-validatie voor classificatie
- **One-Hot encoding** + **Pipelines** zijn standaardgereedschap voor realistische datasets

Om af te sluiten

- <https://app.wooclap.com/MPZDTL?from=event-page>

